

TestPlatform

Rapport d'Execution de Campagne

ms_gestion

Projet: ms_gestion

Version: 2.5

Environnement: ms_gestion_env

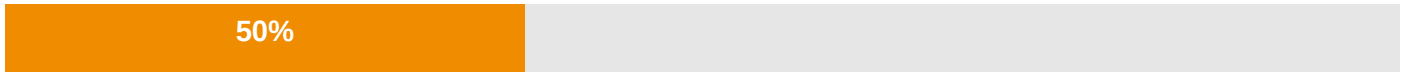
Date: 04/06/2026 10:08

PARTIEL

Resume executif

Total	Succes	Echecs	Erreurs	Duree
2	1	1	0	206.9 s

Taux de reussite



Evolution vs campagne precedente: N/A

Premiere campagne

Informations du contexte

Projet	ms_gestion
Description	test

Suites

Nom	Type	Depot Git	Branche
Suite UNIT	UNIT	N/A	main

Cas de test

Titre	Type	Priorite	Risque	Script
test case add	UNIT	3	CRITICAL	IA
test add () suite test	UNIT	1	CRITICAL	IA

Environnement

URL Web	N/A
URL API	N/A
Repository	https://github.com/ilyeschaabani/Automatisation_intelligente_des_tests_applicatifs
Branche	main
Base de donnees	POSTGRESQL

Resultats details

test case add

SUCCESS

Type	UNIT	Duree	10.0 s
Script	IA	Erreur	[COMPILATION_ERROR] —

test add () suite test

FAILURE

Type	UNIT	Duree	8.3 s
Script	IA	Erreur	[COMPILATION_ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.542 s <<< FAILURE! -- in suites.unit.TestSuiteServiceUnitTest

Analyse IA

test case add

Erreurs extraites	<pre>[INFO] Scanning for projects... [INFO] [INFO] -----< com.pfe.platform:ms_gestion >----- [INFO] Building ms_gestion 0.0.1-SNAPSHOT [INFO] from pom.xml [INFO] -----[jar]----- [INFO] [INFO] --- resources:3.3.1:resources (default-resources) @ ms_gestion --- [INFO] Copying 1 resource from src/main/resources to target/classes [INFO] Copying 0 resource from src/main/resources to target/classes [INFO] [INFO] --- compiler:3.14.1:compile (default-compile) @ ms_gestion --- [INFO] Recompiling the module because of changed source code. [INFO] Compiling 52 source files with javac [debug parameters release 17] to target/classes [INFO] /C:/tmp/ms-execution/exec- 11683413072518821367\Backend/ms_gestion/src/main/java/com/pfe/platform/ms_gestion/service/Campa C:/tmp/ms-execution/exec- 11683413072518821367\Backend/ms_gestion/src/main/java/com/pfe/platform/ms_gestion/service/Campa uses unchecked or unsafe operations. [INFO] /C:/tmp/ms-execution/exec- 11683413072518821367\Backend/ms_gestion/src/main/java/com/pfe/platform/ms_gestion/service/Campa Recompile with -Xlint:unchecked for details. [INFO] [INFO] --- resources:3.3.1:testResources (default-testResources) @ ms_gestion --- [INFO] skip non existing resourceDirectory C:/tmp/ms-execution/exec- 11683413072518821367\Backend/ms_gestion/src/test/resources [INFO] [INFO] --- compiler:3.14.1:testCompile (default-testC</pre>
Analyse IA	1. Ce qui est bien testé : Dans ce script de test, plusieurs tests unitaires sont réalisés sur la couche service de `TestC` utilisant Mockito pour simuler les dépendances du système et isoler le code courant. - Test des méthodes de création de cas de test (`createTestCase`), de lecture d'un cas de test (`getTestCase`) et de suppression d'un cas de test par id (`deleteTestCase`). Ces tests vérifient que la fonctionne comme prévu en utilisant des cas de tests réels et les mocks sont bien injectés dans le contexte. - Les méthodes `findById`, `save` et `deleteById` du repository (`testCaseRepository`) ont également été testées pour vérifier si elles fonctionnent correctement avec les mocks. - Un test d'exception est également présent pour garantir la bonne gestion des cas où l'utilisateur n'est pas autorisé à effectuer une action sur un projet privé. Cela simule le comportement attendu en raison d'une dépendance non réalisée avec Mockito. - Le test vérifie également que `SecurityUtils` est bien utilisé pour obtenir l'utilisateur actuel, qu'il est connecté et que les droits d'accès appropriés sont verrouillés. 2. Suggestion d'amélioration : Pour améliorer le script de test, on peut ajouter plus de tests unitaires pour tous les cas possibles (y compris les exceptions possible, etc.). Cela aidera à s'assurer que le code fonctionne correctement dans toutes les situations. Il est également intéressant d'ajouter des tests d'intégration ou de performance pour voir si le système répond bien aux besoins du client et a une couverture acceptée par les tests unitaires fournis. Une approche peut être pratique de séparer ces tests dans différents fichiers, en fonction des services ou de la complexité du test. L'utilisation d'un framework comme Mockito pour simplifier l'injection des dépendances et le contrôle des mocks simulés dans les tests permet également une meilleure gestion des mocks, ce qui aide à limiter la répétition de code.

test add () suite test

Erreurs extraites	<pre>[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.542 s <<< FAILURE! -- in suites.unit.TestSuiteServiceUnitTest [ERROR] suites.unit.TestSuiteServiceUnitTest.test_add_happyPath -- Time elapsed: 0.032 s <<< FAILURE! java.lang.AssertionError: expected [TestProject] but found [null] [ERROR] Failures: [ERROR] TestSuiteServiceUnitTest.test_add_happyPath:89 expected [TestProject] but found [null] [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 [INFO] BUILD FAILURE [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire- plugin:3.2.5:test (default-test) on project ms_gestion: There are test failures. [ERROR] [ERROR] Please refer to C:\tmp\ms-execution\exec- 11683413072518821367\Backend\ms_gestion\target\surefire-reports for the individual test results. [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream. [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException</pre>
Analyse IA	Cause : Le test est rencontrant une erreur de compilation car Mockito n'a pas pu initialiser correctement les objets mocks. Lors de l'exécution du script, cela signifie que `testSuiteService` n'est pas instancié correctement avec les dépendances des mocks. Correction : La cause racine de l'erreur est la manière dont Mockito est initialisé dans ce test. Il faut utiliser un bloc `try-with-resources` pour garantir que tous les objets mocks sont correctement fermés après chaque test. Dans le script, veuillez ajouter le code suivant avant l'exécution des tests : <pre>```java @BeforeMethod public void setUp() { try (MockedStatic<SecurityUtils> mocked = Mockito.mockStatic(SecurityUtils.class)) { mocks = MockitoAnnotations.openMocks(this); // ... } } ```</pre> Ce bloc `try-with-resources` va garantir que les objets mocks sont correctement fermés après chaque test, ce qui peut résoudre l'erreur de compilation. Conseil : Pour éviter cet type d'échec, assurez-vous que toutes vos dépendances mocks sont correctement initialisées à chaque test avec Mockito et réglez votre code de manière à ce que les objets mocks soient fermés après chaque test.

Evaluation UX

Aucun test UX execute pour cette campagne

Metriques

Top 5 des tests les plus lents

Test	Suite	Duree
test case add	Suite UNIT	10.0 s
test add () suite test	Suite UNIT	8.3 s

Temps d'execution par type

Type	Duree moyenne
UNIT	9.1 s

Tests instables

Aucun test instable detecte

Recommandations

- test add () suite test:

Cause : Le test est rencontrant une erreur de compilation car Mockito n'a pas pu initialiser correctement les objets mocks. Lors de l'exécution du script, cela signifie que `testSuiteService` n'est pas instancié correctement avec les dépendances des mocks.

Correction : La cause racine de l'erreur est la manière dont Mockito est initialisé dans ce test. Il faut utiliser un bloc `try-with-resources` pour garantir que tous les objets mocks sont correctement fermés après chaque test. Dans le script, veuillez ajouter le code suivant avant l'exécution des tests :

```
```java
@BeforeMethod
public void setUp() {
 try (MockedStatic<SecurityUtils> mocked = Mockito.mockStatic(SecurityUtils.class)) {
 mocks = MockitoAnnotations.openMocks(this);
 // ...
 }
}
```
```

Ce bloc `try-with-resources` va garantir que les objets mocks sont correctement fermés après chaque test, ce qui peut résoudre l'erreur de compilation.

Conseil : Pour éviter cet type d'échec, assurez-vous que toutes vos dépendances mocks sont correctement initialisées à chaque test avec Mockito et réglez votre code de manière à ce que les objets mocks soient fermés après chaque test.

Annexe - Scripts generes par IA

test case add

```
package suites.unit;

import com.pfe.platform.ms_gestion.dto.request.CreateTestCaseRequest;
import com.pfe.platform.ms_gestion.dto.response.TestCaseResponse;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.TestCase;
import com.pfe.platform.ms_gestion.entity.TestSuite;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.TestCaseRepository;
import com.pfe.platform.ms_gestion.repository.TestSuiteRepository;
import com.pfe.platform.ms_gestion.security.SecurityUtils;
import com.pfe.platform.ms_gestion.service.LlmService;
import com.pfe.platform.ms_gestion.service.TestCaseService;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.MockitoAnnotations;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
import java.util.Optional;

public class TestCaseServiceTest {

    @Mock private TestCaseRepository testCaseRepository;
    @Mock private TestSuiteRepository testSuiteRepository;
    @Mock private ProjectMemberRepository projectMemberRepository;
    @Mock private LlmService llmService;
    @InjectMocks private TestCaseService testCaseService;

    private AutoCloseable mocks;
    private MockedStatic<SecurityUtils> mockedSecurity;

    @BeforeMethod
    public void setUp() {
        mocks = MockitoAnnotations.openMocks(this);
        mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
        mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mockedSecurity.close();
        mocks.close();
    }

    @Test
    public void testAddTestCase() {
        Long suiteId = 1L;

        Project project = new Project();
        project.setId(10L);
        project.setName("Test Project");

        ProjectMember member = new ProjectMember();
        member.setRole(ProjectMember.Role.ADMIN);
        member.setUserId(1L);
        member.setProject(project);

        TestSuite suite = new TestSuite();
```

```

suite.setId(suiteId);
suite.setType(TestSuite.TestType.UNIT);
suite.setProject(project);

CreateTestCaseRequest request = new CreateTestCaseRequest();
request.setTitle("Login test");
request.setType("UNIT");
request.setRiskLevel("LOW");
request.setPriority(1);
request.setDescriptionAI("Test the add method");

Mockito.when(testSuiteRepository.findById(suiteId))
    .thenReturn(Optional.of(suite));
Mockito.when(projectMemberRepository.findByIdAndUserId(
    Mockito.any(), Mockito.eq(1L)))
    .thenReturn(Optional.of(member));

Mockito.when(llmService.generateTestCode(
    Mockito.any(), Mockito.any(), Mockito.any()))
    .thenReturn("public class FakeTest {}");

Mockito.when(testCaseRepository.save(Mockito.any()))
    .thenAnswer(inv -> {
        TestCase tc = inv.getArgument(0);
        tc.setSuite(suite);
        return tc;
    });

TestCaseResponse response = testCaseService.add(suiteId, request);

Assert.assertNotNull(response);
Assert.assertEquals(response.getTitle(), "Login test");
Assert.assertEquals(response.getRiskLevel(), "LOW");
Mockito.verify(testCaseRepository, Mockito.times(1)).save(Mockito.any());
// verify
Mockito.verify(llmService, Mockito.times(1))
    .generateTestCode(Mockito.any(), Mockito.any(), Mockito.any());
}
}

```

test add () suite test

```

package suites.unit;

import com.pfe.platform.ms_gestion.dto.request.CreateTestSuiteRequest;
import com.pfe.platform.ms_gestion.dto.response.TestSuiteResponse;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.TestSuite;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.ProjectRepository;
import com.pfe.platform.ms_gestion.repository.TestSuiteRepository;
import com.pfe.platform.ms_gestion.security.SecurityUtils;
import com.pfe.platform.ms_gestion.service.TestSuiteService;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.MockitoAnnotations;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

import java.util.Optional;

public class TestSuiteServiceUnitTest {

```

```
@Mock private TestSuiteRepository testSuiteRepository;
@Mock private ProjectRepository projectRepository;
@Mock private ProjectMemberRepository projectMemberRepository;

@InjectMocks
private TestSuiteService testSuiteService; // pas de new()

private AutoCloseable mocks;
private MockedStatic<SecurityUtils> mockedSecurity;

@BeforeMethod
public void setUp() {
    mocks = MockitoAnnotations.openMocks(this);
    mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
    mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
}

@AfterMethod
public void tearDown() throws Exception {
    mockedSecurity.close();
    mocks.close();
}

@Test
public void test_add_happyPath() {
    // Arrange
    Long projectId = 1L;

    Project project = new Project();
    project.setId(projectId);

    ProjectMember member = new ProjectMember();
    member.setUserId(1L);
    member.setRole(ProjectMember.Role.ADMIN);
    member.setProject(project);

    TestSuite saved = new TestSuite();
    saved.setId(10L);
    saved.setProject(project);

    // Mock repositories
    Mockito.when(projectRepository.findById(projectId))
        .thenReturn(Optional.of(project));
    Mockito.when(projectMemberRepository.findByProjectIdAndUserId(
        Mockito.eq(projectId), Mockito.eq(1L)))
        .thenReturn(Optional.of(member));
    Mockito.when(testSuiteRepository.save(Mockito.any()))
        .thenReturn(saved);

    // Request avec les test data générés
    CreateTestSuiteRequest request = new CreateTestSuiteRequest();
    request.setName("TestProject");
    request.setType("UNIT");
    request.setGitRepoUrl("https://github.com/testproject.git");
    request.setGitBranch("master");
    request.setModulePath("/src/main/java/com/example/test");

    // Act
    TestSuiteResponse response = testSuiteService.add(projectId, request);

    // Assert
    Assert.assertNotNull(response);
    Assert.assertEquals(response.getName(), "TestProject");
    Mockito.verify(testSuiteRepository, Mockito.times(1)).save(Mockito.any());
    Mockito.verify(projectRepository, Mockito.times(1)).findById(projectId);
}
```